PYTHON ACTIVITY RECORD

Repeated Word and Phrase Detection

1. Problem Statement

Write a Python program that accepts a paragraph, converts it to lowercase, removes selected punctuation marks, and identifies repeated individual words. For each repeated word, display the number of occurrences and the positions where it appears.

2. Objectives

- Accept a paragraph from the user.
- Convert the paragraph to lowercase to make matching case-insensitive.
- Replace selected punctuation marks with spaces.
- Split the paragraph into words.
- Find each word that appears more than once.
- Display the repeated word, its count, and its 1-based positions, or display a message when no repeated words are found.

3. Assumptions

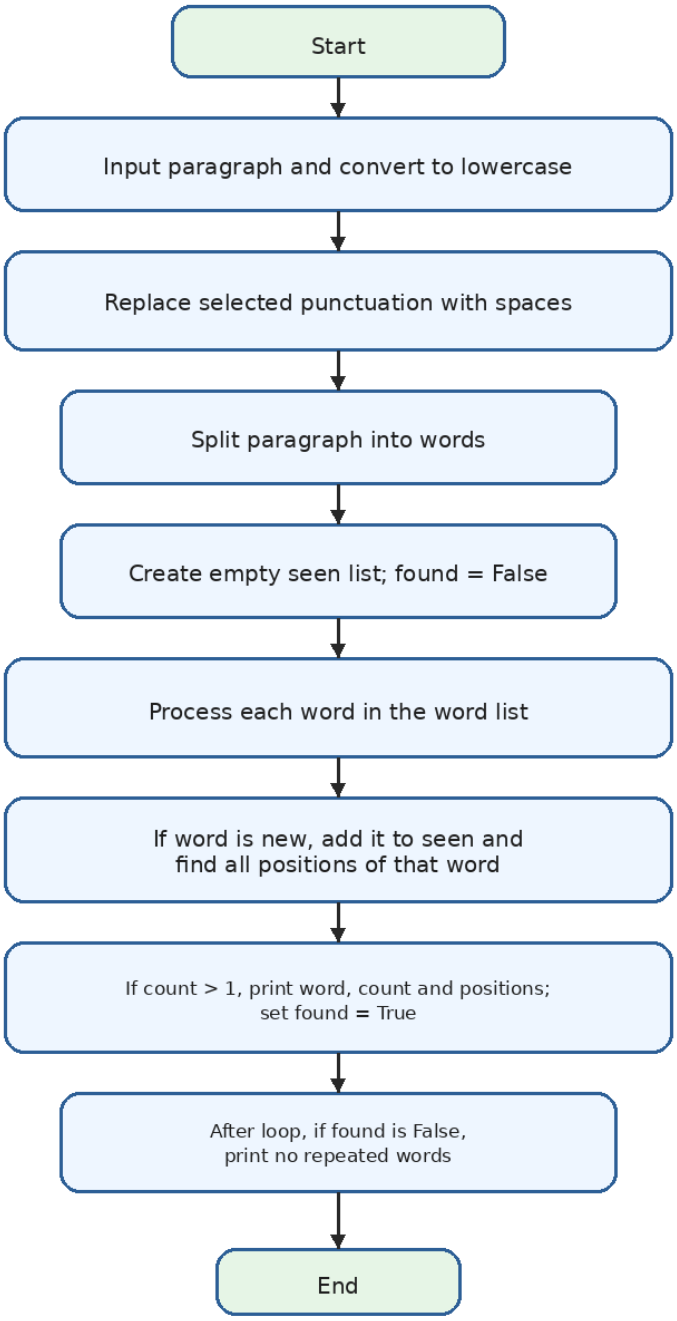
- The user enters plain text as a paragraph.
- Word positions are numbered from 1, starting with the first word.
- Uppercase and lowercase versions are treated as the same word.
- Only . , ! ? ; : and parentheses are replaced with spaces.
- The provided code detects repeated single words, not multi-word phrases, even though the filename mentions phrases.

4. Algorithm

1. Start the program.
2. Input the paragraph and convert it to lowercase.
3. Replace the selected punctuation marks with spaces.
4. Split the paragraph into a list of words.
5. Create an empty list named seen and set found to False.
6. For each word, check whether it is already in seen.
7. If the word is new, add it to seen and collect all its positions in the word list, starting from 1.
8. If the word occurs more than once, display its count and positions and set found to True.
9. After the loop, if found is False, display “No repeated words found.”
10. Stop.

5. Flowchart

The flowchart represents the processing steps used by the supplied program.



6. Pseudocode

```
BEGIN
    INPUT paragraph
    CONVERT paragraph to lowercase

    FOR each mark in ".,!?;:()"
        REPLACE mark with a space
    END FOR

    words = split paragraph into words
    seen = empty list
```

```
    found = False

    FOR each word in words
        IF word is not in seen THEN
            ADD word to seen
            positions = all 1-based positions of word in words
            IF length(positions) > 1 THEN
                DISPLAY word, count, positions
                found = True
            END IF
        END IF
    END FOR

    IF found is False THEN
        DISPLAY "No repeated words found."
    END IF
END
```

7. Python Source Code

```
paragraph = input("Enter paragraph: ").lower()

for mark in ".,!?;:()":

    paragraph = paragraph.replace(mark, " ")

words = paragraph.split()

seen = []

found = False

for word in words:

    if word not in seen:

        seen.append(word)

        positions = [i + 1 for i, item in enumerate(words) if item == word]

        if len(positions) > 1:

            found = True

            print(word, "count:", len(positions), "positions:", positions)

if not found:

    print("No repeated words found.")
```

8. Test Cases

Test	Input paragraph	Expected output	Purpose
1	The cat sat, and the cat slept. CAT!	the count: 2 positions: [1, 5]; cat count: 3 positions: [2, 6, 8]	Repeated words and case-insensitive matching
2	Python makes coding easy.	No repeated words found.	Unique words only
3	(empty input)	No repeated words found.	Empty paragraph

9. Output Screenshots

Test Case 1: Repeated words

```
Python Output - Repeated words
Enter paragraph: The cat sat, and the cat slept. CAT!
the count: 2 positions: [1, 5]
cat count: 3 positions: [2, 6, 8]
```

Test Case 2: No repeated words

```
Python Output - No repeated words
Enter paragraph: Python makes coding easy.
No repeated words found.
```

Test Case 3: Empty input

```
Python Output - Empty input
Enter paragraph:
No repeated words found.
```

10. Conclusion

The program demonstrates string handling, lists, loops, and conditional statements in Python. It finds repeated individual words, counts their occurrences, and reports their positions. The program ignores capitalization and removes selected punctuation. It does not detect repeated multi-word phrases.